

SQL as a Domain-Specific Language for Demographic Microsimulation

Abstract

Demographic microsimulation systems traditionally require substantial programming effort, with each demographic process implemented as custom code that must be compiled or interpreted within a general-purpose language. This paper proposes leveraging SQL as a domain-specific language (DSL) for demographic microsimulation, drawing inspiration from the historical relationship between Lisp and DSLs. We demonstrate that by using SQL as both a specification language and an execution engine, we can create a system where demographic models are defined as data (SQL queries) rather than code. This approach, implemented in the Talos microsimulation engine, enables researchers to specify and modify models without programming expertise while maintaining the performance benefits of a compiled execution environment.

1. Introduction

Microsimulation models have become essential tools for demographic analysis and policy evaluation. These models simulate individual-level transitions across demographic states—aging, mortality, fertility, education, income dynamics—and aggregate these individual histories to produce population projections . The complexity of these models has grown substantially. Contemporary implementations, such as the MikroSim model for Germany, incorporate numerous modules with complex interdependencies, requiring careful ordering of transitions to maintain behavioral realism .

Despite their analytical power, microsimulation systems face a persistent challenge: the gap between model specification and implementation. Demographic researchers typically conceive models in terms of transition probabilities, conditional dependencies, and state changes—concepts that map naturally to declarative specifications. However, most implementations require translating these specifications into imperative code (R, Python, C++), creating barriers to model development and modification .

2. Domain-Specific Languages and the Lisp-SQL Connection

2.1 The Nature of Domain-Specific Languages

A domain-specific language is a programming language designed to solve a finite class of problems . Where general-purpose languages like Python or C++ provide maximum flexibility, DSLs offer focused expressiveness within a particular domain. The history of computing offers numerous examples: TeX for typesetting, MATLAB for numerical computation, and SQL itself for database manipulation .

2.2 Lisp's Enduring Contribution: Code as Data

Lisp introduced a revolutionary concept: the program and the data it manipulates share the same representation. This property, known as homoiconicity, enables **metaprogramming**—writing programs that write programs. Lisp macros allow developers to create custom syntax structures that match problem domains without modifying the underlying compiler .

The significance for DSLs is profound. As Wadler observes, Lisp's approach to domain-specific languages through quoted terms, introduced in 1960, has found modern expression in systems like Microsoft's LINQ and Feldspar . The key insight is that quoted terms allow the DSL to share the syntax and type system of the host language while normalizing those terms to ensure the subformula principle—guaranteeing, for example, that higher-order features in the source produce first-order code in the target .

2.3 SQL as Lisp's Logical Descendant

The connection between Lisp and SQL is more than historical parallel. As Edmunds notes, Lisp implementations for relational databases often provide a "Lisp-y syntax for SQL queries" . This is not mere syntactic sugar but reflects a deeper architectural affinity: both Lisp and SQL are declarative, both operate on structured data, and both can be interpreted or compiled.

More importantly, SQL represents a successful DSL that has been optimized for data manipulation over decades. Its grammar, while finite, captures a rich set of operations: selection, projection, join, aggregation, window functions, and recursive queries. Modern implementations extend this with procedural extensions (PL/SQL, T-SQL) that transform SQL into a hybrid DSL for data-intensive computation.

3. The SQL Microsimulation Architecture

3.1 Models as SQL Statements

The core insight of this approach is that demographic transitions can be expressed as SQL **UPDATE** statements. Consider a mortality transition:

```
UPDATE population
SET alive = false
WHERE age >= 85 AND random() < 0.10
```

This statement captures the essential logic: individuals meeting a condition face a probability of death. The SQL engine handles the iteration, conditional evaluation, and state update.

More complex transitions are equally expressible:

```
UPDATE population p
SET education_level = 'tertiary'
WHERE p.age BETWEEN 18 AND 25
AND p.education_level = 'secondary'
AND random() < (
  SELECT enrollment_rate
  FROM education_rates
  WHERE year = {CURRENT_YEAR}
)
```

Here, a subquery retrieves contextual parameters, demonstrating how SQL's relational model naturally integrates external data sources—a key requirement in microsimulation where transition probabilities often depend on external tables .

3.2 The SQL Engine as Interpreter

This architecture leverages the SQL engine not merely as a storage layer but as a **domain-specific interpreter**. Each model definition is a data element (a string containing SQL) that the system executes. This mirrors the Lisp philosophy where code and data share representation: the model specification is both human-readable documentation and executable program.

Modern SQL engines provide sophisticated optimization and execution capabilities. As jOOQ's SQL parser demonstrates, SQL statements can be parsed into abstract syntax trees,

transformed, and rendered for different dialects . The expressQL project further demonstrates that SQL-like expressions can be parsed and composed programmatically, creating a bridge between natural SQL syntax and programmatic construction .

3.3 Parameterization and Model Configuration

The approach becomes practical through **parameterization**. Rather than hardcoding transition probabilities, models reference parameters:

```
UPDATE population
SET alive = false
WHERE age < 1 AND random() < {mortality_rates.infant}
```

A preprocessing step substitutes these placeholders with values from a configuration file. This separates model structure from model parameters—a researcher can adjust mortality rates without modifying the SQL statement, and can create new models by composing existing patterns.

3.4 Module Ordering and Dependencies

Microsimulation systems require careful ordering of transitions. As noted in the MikroSim model description, "not the order of modules but the modeling strategy is crucial for the simulation" . The SQL approach accommodates this through a priority-based execution system:

```
models:
- name: "age_increment"
  priority: 1
  query: "UPDATE population SET age = age + 1"
- name: "mortality"
  priority: 2
  query: "UPDATE population SET alive = false WHERE ..."
```

This explicit ordering allows modelers to specify dependencies declaratively, similar to how MikroSim orders its modules (Mortality, Births, Regional Mobility) to ensure consistent state transitions .

4. Advantages and Implications

4.1 Accessibility and Model Transparency

The SQL approach dramatically lowers barriers to model development. Demographers familiar with SQL (or learnable in hours) can specify models directly, without requiring programming expertise. The specification remains executable and auditable—the model is the code.

4.2 Performance

SQL engines are highly optimized for set-based operations. A single SQL UPDATE can process millions of records efficiently, using indexes, vectorized execution, and query optimization. In contrast, row-by-row simulation in R or Python often becomes performance-bound for large populations. Modern SQL implementations also support sophisticated optimizations: jOOQ's SQL parser, for instance, can translate queries across dialects and even interpret them incrementally.

4.3 Model Reuse and Composition

SQL's relational algebra provides natural composition mechanisms. Models can reference derived tables, use subqueries, or join across data sources. This enables the "ability to link data sets together" that is "the true strength of microsimulation".

4.4 Implementation Considerations

The approach is feasible with any SQL engine. The Talos implementation uses a pure Go SQLite engine, eliminating external dependencies and enabling single-binary deployment. SQLite's in-memory capability supports fast iteration while maintaining full SQL expressiveness.

5. Conclusion

The use of SQL as a domain-specific language for demographic microsimulation draws on a rich heritage of DSL design. From Lisp's early insight that code and data can share representation to the modern SQL parser ecosystems, this approach offers a practical pathway to model-centric microsimulation. By defining models as SQL statements, researchers can specify, modify, and compose demographic transitions declaratively, leveraging decades of optimization in SQL engines while maintaining the flexibility to express complex behaviors. The result is a system where models become data—auditable, parameterizable, and executable—embodying the Lisp principle of "code as data" in the

context of demographic simulation.

References

- [1] "Definition: special-purpose language," ComputerLanguage.com.
- [2] "Building Domain-Specific Languages - Unleashing Lisp: Power of Symbolic Programming," StudyRaid.
- [3] P. Wadler, "Everything old is new again: Quoted domain specific languages," International Summer School on Metaprogramming, Cambridge, 2016.
- [4] E. Weitz, "Common Lisp Recipes," Chapter 21: Persistence.
- [5] D. Ballas et al., "SimLeeds: A Microsimulation Model for Leeds," Table 3 attributes.
- [6] R. Münnich et al., "A Population Based Regional Dynamic Microsimulation," MDA Journal, 2021.
- [7] W. Garcia et al., "A Microsimulation Model for Population Projection in Colombia," PAA 2018.
- [8] A. Brennan et al., "Dynamic Microsimulation of Social Roles," International Journal of Microsimulation, 2020.
- [9] "expressQL - A Pythonic DSL for SQL Conditions," PyPI.
- [10] jOOQ Manual 3.19, "SQL Parser and Interpreter."
- [11] "Execution Optimization of Database Queries," US Patent 11,461,324 B2.
- [12] jOOQ Manual 3.17, "SQL Parser API."